From Bounded Model Checking to Mechanized Proof: A Multi-Level Verification Case Study for a Price-Time Priority Matching Engine*

Ara Aslyan
Strike Technologies, New York
aaslyan@striketechnologies.com

04/11/2026

Abstract

We present a multi-level formal verification case study of a feature-rich price-time priority matching engine. The specification covers a full order-type suite (LIMIT, MARKET, MARKET-TO-LIMIT, STOP_LIMIT, STOP_MARKET) together with iceberg orders, four self-trade prevention policies, post-only, Fill-Or-Kill, minimum-quantity, cancel, and amend semantics. Verification proceeds at two levels: bounded exhaustive state-space exploration of a TLA+ model, and a mechanized proof in Lean 4 of a Lean reference implementation that mirrors the same specification.

Within the explored TLC configurations we identify one concrete specification defect—a FIFO violation caused by triggered stop orders retaining their original timestamp—and one specification gap in the interaction between STP DECREMENT and iceberg visibility management. In Lean, we prove that the main processing theorem preserves the combined book invariant `AllInv`, defined as uncrossed plus sorted on both sides. The proof holds for arbitrary book sizes with a fuel bound computed from the book state and proved sufficient rather than assumed. The development consists of a 3,986-line constructive proof of the full preservation theorem, together with a 312-line structural proof of the matching-and-dispose core via a three-case analysis of the termination condition.

The proof effort itself surfaced a further issue that neither prose review nor bounded model checking had exposed: the original arbitrary fuel bound (`defaultFuel = 100`) made the preservation theorem false for books with more than a hundred contra-side orders. We report this as a methodological observation on the verification pipeline.

1 Introduction

Matching-engine correctness is commercially important and surprisingly difficult to state precisely. Production engines combine a large number of interacting features—order types, time-in-force policies, self-trade prevention, hidden quantity, stop triggers, cascading fills—each specified correctly in isolation, but capable of producing unexpected emergent behavior when combined. Prose specifications resolve the easy cases and leave the hard ones to implementor judgment.

Formal methods offer a sharper alternative: write the semantics as an executable state machine and enumerate its reachable states. This paper is a case study in doing exactly that for a matching-engine specification, and then going one step further—lifting the same semantics into Lean 4 and proving a safety property that bounded model checking cannot, by itself, establish.

The paper's contributions are:

*Artifacts available at https://github.com/formally-verified-exchange/verified-matching-engine.

1. A TLA+ formalization of a feature-rich price-time-priority matching-engine specification, together with a suite of invariants covering book consistency, queue fairness, and order-type safety.
2. A concrete TLC counterexample showing that triggered stop orders can violate FIFO if activation does not refresh queue priority, and identification of a second defect in the interaction between STP DECREMENT and iceberg visibility management.
3. A mechanized Lean 4 proof that the processing pipeline preserves the combined book invariant `AllInv` (uncrossed plus sorted on both sides) under a minimal well-formedness precondition, for arbitrary book sizes, with the matching-loop fuel bound computed from the book state and proved sufficient rather than assumed.
4. Two complementary proof artifacts with no open obligations: a 3,986-line constructive proof of the full preservation theorem by branch-by-branch induction, and a 312-line structural proof of the matching-and-dispose core by three-case analysis of the matching termination condition. The contrast illustrates two complementary proof styles.
5. A methodological observation from the proof effort itself: the original arbitrary fuel bound (`defaultFuel = 100`) made the preservation theorem false for sufficiently populated books.
6. A discussion of specification-guided implementation generation, framed as an engineering observation about how a precise formal specification constrains downstream coding, *not* as a claim of end-to-end refinement between the Lean reference and any systems-language implementation.

1.1 Note on the Development Process

The TLA+ model, the Lean 4 proofs, and the C/C++ implementations discussed in §8 were developed interactively with a large language model acting as a coding assistant. Strategic decisions—the progress-lemma decomposition, the three-case analysis in the elegant proof, and the side-parameterization refactor—were human contributions. The bulk of tactic scripting and operator translation was produced by the model under that guidance. We mention this because the per-file size and per-lemma count reported in later sections reflect that mode of work; the methodology described in this paper is independent of whether the implementer is a single engineer, a team, or an AI-augmented workflow.

1.2 Paper Organization

The paper is organized as follows. Section 2 describes the matching-engine semantics under study. Section 3 presents the TLA+ model and the scope of the bounded exploration. Section 4 enumerates the invariants checked. Section 5 presents the two TLC findings with counterexample traces, and Section 6 gives the corresponding specification fixes. Section 7 presents the Lean 4 proof of `AllInv` preservation: the termination measure, the progress-lemma decomposition, the mutual induction across the processing pipeline, and a second, shorter proof via case analysis of the matching termination condition. An observation from the proof effort itself—the constant-fuel soundness issue—is reported in the same section. Section 8 describes how the fixed specification was carried into C and C++ implementations and why we stop short of a refinement claim. Section 9 collects lessons on feature composition; Section 11 positions the work against prior literature; Section 10 states limitations and reproducibility conditions; Section 12 concludes.

2 Matching-Engine Semantics Under Study

2.1 Specification Source

The object of study is an internal prose specification, `matching-engine-formal-spec.md` version 1.2.0, describing a generic asset-agnostic order matching engine with price-time priority (FIFO). The specification defines primitive domains, order well-formedness constraints, the order book structure, a core matching algorithm, post-match disposition, six order types, four STP policies, iceberg reload semantics, stop triggering with cascading evaluation, cancel, and amend.

The specification is 900 lines of structured pseudocode. It was written with care—well-formedness constraints are enumerated, invariants are stated formally, and edge cases such as MTL price derivation and FOK dry-run semantics are treated explicitly. It is not a toy specification.

2.2 Order Types and Features

The engine supports five order types:

- **LIMIT**: rests at a specified price; fills at that price or better.
- **MARKET**: fills at any available price; never rests on the book.
- **MARKET-TO-LIMIT (MTL)**: aggressively matches at any price; locks to the first execution price and rests any remainder as a LIMIT order.
- **STOP_LIMIT / STOP_MARKET**: dormant until a trigger condition on the last trade price is satisfied, then converted to LIMIT or MARKET respectively and re-entered into the matching pipeline.

Order modifiers include:

- **Iceberg** (`displayQty`): only $visibleQty \leq displayQty$ is shown to the market; after the visible slice is consumed, the order reloads with a fresh visible slice and loses time priority.
- **Post-Only**: the order must not aggress; rejected (under the REJECT policy) if it would cross the spread.
- **TimeInForce**: GTC, GTD, IOC, FOK, DAY.
- **MinQty**: requires a minimum fill on first execution; the constraint is cleared after the initial threshold is met, allowing the remainder to rest.
- **Self-Trade Prevention (STP)**: four policies (CANCEL_NEWEST, CANCEL_OLDEST, CANCEL_BOTH, DECREMENT) govern what happens when an incoming order would trade against a resting order in the same STP group. The incoming order's policy governs.

2.3 Well-Formedness Constraints

The prose specification enumerates 23 well-formedness constraints (WF-1 through WF-20 plus three a/b suffixed refinements) that define the *valid inputs* to the engine. Every theorem in the Lean proof, every TLC invariant, and every rejection message in the synthesized C++ code traces back to a subset of these constraints. They are grouped by feature and summarized in Table 1. The full text with exact quantifiers appears in `matching-engine-formal-spec.md §2.4`.

Table 1: Well-formedness constraints (excerpt). These are the preconditions of every theorem in the Lean proof and the admission criteria for the TLA+ `SubmitOrder` action.

ID	Group	Constraint
WF-1	Qty	$quantity > 0$
WF-2/2a/2b	Price	LIMIT, STOP_LIMIT require $price > 0$; MTL has $price = \bot$
WF-3	Price	MARKET, STOP_MARKET have $price = \bot$
WF-4	Stop	Stop orders require $stopPrice > 0$
WF-5	Stop	Non-stop orders have $stopPrice = \bot$
WF-6/7	TIF	GTD requires $expireTime > now$; otherwise $expireTime = \bot$
WF-8/8a	TIF	MARKET⇒IOC/FOK; MTL⇒GTC/GTD/DAY
WF-9/10	Iceberg	$0 < displayQty \leq quantity$; icebergs must be LIMIT
WF-11/12	Iceberg	Initial $remainingQty = quantity$; $visibleQty = \min(displayQty, quantity)$
WF-13	PostOnly	Post-only⇒LIMIT
WF-14	PostOnly	Post-only$\not\Rightarrow$IOC/FOK
WF-15	PostOnly	MARKET, MTL⇒¬post-only
WF-16	STP	$selfTradeGroup = \bot \iff stpPolicy = \bot$
WF-17	STP	$stpPolicy \in \{$CANCEL_NEWEST, CANCEL_OLDEST, CANCEL_BOTH, DECREMENT$\}$
WF-18	MinQty	$0 < minQty \leq quantity$
WF-19	MinQty	MinQty⇒¬post-only
WF-20	MinQty	FOK⇒$minQty = \bot$

The Lean predicate `OrderProcOk` used in the main theorem is the minimum subset of these constraints that the matching loop actually depends on. It does not admit every WF-valid order—it admits every WF-valid order that can enter the matching phase—and its two conjuncts are refinements of WF-13 (post-only⇒LIMIT, and hence post-only orders carry a defined price) and WF-15 (STOP_MARKET⇒¬post-only). The fact that these two minimal fragments—and no more—suffice was discovered *during* the proof: earlier attempts omitted them and every branch of the matching loop that case-split on an undefined price failed. The proof told us which WF clauses were load-bearing.

2.4 Core Invariants Stated in the Specification

The prose specification states 23 invariants that must hold after every `PROCESS` call completes (§3 and §13 of the spec). Table 2 gives the book-structural core; the complete TLC-checked inventory including safety and progress properties is given in Table 5 (§4).

Table 2: Core book invariants (excerpt).

ID	Group	Statement
INV-1	Structural	Every price level has a non-empty order queue
INV-2	Structural	$\forall$ consecutive bid levels (L_1, L_2): $L_1.price > L_2.price$
INV-3	Structural	$\forall$ consecutive ask levels (L_1, L_2): $L_1.price < L_2.price$
INV-4	Structural	$bestBid < bestAsk$ or either side empty (**uncrossed**)
INV-5	Content	Every resting order has $remainingQty > 0$
INV-6	Content	Every resting order has status $\in \{\text{NEW}, \text{PARTIALLY_FILLED}\}$
INV-7	Ordering	FIFO within each price level: consecutive orders have strictly increasing timestamps
INV-8	Content	No MARKET orders rest on the book
INV-9	Trade	Every trade executes at the passive order's price
INV-10	Events	Events emitted in causal order
INV-11	PostOnly	No trade where the aggressor was post-only
INV-12	STP	No trade between orders in the same STP group
INV-13	Content	No MTL orders rest on the book
INV-14	Content	No $minQty$ is set on any resting order

The FIFO invariant INV-7 is central to this paper's first bug. Its formal statement is:

For every price level, consecutive orders satisfy $o_1.timestamp < o_2.timestamp$.

This is the formal statement of price-time priority. A violation means an order with a later arrival time has higher queue position than one that arrived earlier—the queue is out of order.

2.5 The `canMatch` Predicate

A single predicate governs the matching loop's continuation:

$$\texttt{canMatchPrice}(o, p) \equiv \begin{cases} \top & o.type \in \{\text{MARKET}, \text{STOP_MARKET}\} \\ o.price \geq p & o.side = \text{BUY},\ o \text{ priced} \\ o.price \leq p & o.side = \text{SELL},\ o \text{ priced} \\ \top & o.type = \text{MTL},\ \text{no trades yet} \\ \text{derived price} & o.type = \text{MTL},\ \text{after first fill} \end{cases}$$

This predicate is the loop guard of `doMatch`. When the loop terminates in a non-empty contra side, it terminates *because* $\neg\texttt{canMatchPrice}(o, bestContra)$. This observation is the seed of the elegant proof in §7.7: if matching stopped because prices no longer cross, then the book cannot be crossed after disposition places the residual on the same side as the aggressor.

2.6 From Specification to Proof: Traceability

The specification is not documentation—it is the shared vocabulary that binds TLC and Lean. Every numbered clause appears in at least one of the two. Table 3 makes the chain explicit for the clauses exercised in this paper.

Table 3: Spec → TLA+ → Lean traceability for the clauses touched by the main theorem. The Lean column refers to `Theorems.lean` or `Invariants.lean`.

Spec clause	TLA+ artefact	Lean artefact	Role
WF-13	`ValidOrder` guard	`OrderProcOk` field 1	Main-theorem precondition
WF-15	`ValidOrder` guard	`OrderProcOk` field 2	Main-theorem precondition
INV-2/3	not separately checked[†]	`bidsSortedDesc`, `asksSortedAsc`	Discovered: needed for induction
INV-4	`BookUncrossed`	`BookUncrossed`, part of `AllInv`	Main-theorem conclusion
INV-5	`NoGhosts`	`NoGhosts` (definition only)	Not proved preserved
INV-6	`StatusConsistency`	`NoGhosts` (derived)	Not explicitly threaded in Lean
INV-7	`FIFOWithinLevel`	—	TLC-only
INV-8	`NoRestingMarkets`	—	TLC-only
INV-11	`PostOnlyGuarantee`	—	TLC-only
INV-12	`STPGuarantee`	—	TLC-only
INV-13	`NoRestingMTL`	—	TLC-only
INV-14	`NoRestingMinQty`	`NoRestingMinQty`	Invariants.lean definition
`canMatch`	`CanMatchPrice`	`canMatchPrice`	Matching loop guard
PROCESS §12	`ProcessOrder` (11 phases)	`processOrder` (11 phases)	Mutual induction target

[†]Structural sortedness is not stated as a separate TLC invariant because TLA+ sequences are ordered by construction from a sorted key. In Lean the sortedness is explicit because book levels are a `List PriceLevel` and the induction needs to know both sides are monotone. This mismatch was discovered *during* the proof: `BookUncrossed` alone is not inductive, which forced the addition of `AllInv = BookUncrossed` ∧ `bidsSortedDesc` ∧ `asksSortedAsc`. The proof refined the invariant.

The lesson is that invariants are *discovered*, not merely declared. The first row of surprises in Table 3 (sortedness) came from the Lean induction refusing to close; `AllInv` is what the proof requires, not what the prose spec emphasizes. The second row of surprises (`OrderProcOk` as a minimal precondition fragment) came from the proof refusing to close without WF-13 and WF-15. In both cases the proof told us the specification was under-emphasizing something that was, on reflection, quite obvious.

3 TLA+ Model and Bounded Exploration

3.1 Model Structure

The TLA+ specification (`MatchingEngine.tla`, approximately 850 lines) is a direct translation of the prose specification. The state consists of eight variables:

```
bidQ           : PRICES → Seq(Order)   (* per-price bid queues *)
askQ           : PRICES → Seq(Order)   (* per-price ask queues *)
stops          : Set(Order)             (* dormant stop orders *)
lastTradePrice : PRICES \cup {NULL}     (* last execution price *)
postOnlyOK     : Bool                   (* no post-only aggressor
   trade *)
stpOK          : Bool                   (* no same-group trade *)
nextId         : Nat                    (* next order ID *)
clock          : Nat                    (* logical clock *)
```

The model has four actions: `SubmitOrder`, `CancelOrder`, `AmendOrder`, and `TimeAdvance`. `SubmitOrder` nondeterministically selects all valid order parameters and invokes the full pro-

cessing pipeline. TLC's exhaustive exploration therefore considers every valid combination of order type, side, price, quantity, TIF, STP policy, iceberg flag, post-only flag, and minimum quantity, within the configured bounds.

3.2 Processing Pipeline

The pipeline operator `ProcessOrder` implements all phases of the prose specification's `PROCESS` procedure, including recursive operators for the matching loop (`DoMatch`) and stop-trigger cascade (`ProcessCascade`, `ProcessTriggeredStops`). Mutual recursion between these operators is required to model cascading stop triggers correctly: a triggered stop may generate trades, which may trigger further stops.

A `fuel` parameter on `DoMatch` bounds recursion depth for TLC termination safety. Under the small quantity bounds used in model checking, this bound is never reached in practice.

3.3 Scope of Exploration

The model is parameterized by `PRICES`, `MAX_QTY`, `MAX_ORDERS`, and `MAX_CLOCK`. Configurations explored are shown in Table 4. All configurations include the full order-type suite, all four STP policies, icebergs, post-only, FOK, MinQty, and DAY expiry. The "With Amend" configuration adds the `AmendOrder` action.

Table 4: Model checking configurations and statistics.

Configuration	Orders	Qty	Prices	Amend	States Gen.	Distinct	Runtime	Result
Tiny	2	1	{1,2}	No	2,979,719	1,123,333	20 s	Pass
Small	2	2	{1,2}	No	11,528,343	6,037,674	1 m 42 s	Pass
Medium	2	2	{1,2,3}	No	36,700,016	21,261,901	5 m 34 s	Pass
With Amend	2	2	{1,2}	Yes	25,209,607	9,104,902	3 m 33 s	Pass
3-order	3	2	{1,2}	No	>45 M	>26 M	>10 m	*partial*

All "Pass" results above are post-fix. Prior to fixing the stop-timestamp bug, the Small configuration produced a counterexample on the `FIFOWithinLevel` invariant within the first 11.5 million states. The 3-order run was interrupted before state-space exhaustion; "partial" means no violation was found in the explored states, not clean passage under all 3-order behaviors.

3.4 What the Model Does Not Cover

- The model uses a single STP group identifier (`"G1"`). Multiple independent groups are not explored.
- GTD expiry is modeled via `TimeAdvance` but without wall-clock constraints.
- Event ordering (INV-10) and market-data publication are not modeled.
- The model checks specification consistency, not implementation correctness.

4 Invariants Checked

All invariants are checked as TLA+ safety properties (`INVARIANT` declarations in the TLC configuration). TLC verifies that each invariant holds in every reachable state within the explored configurations.

Table 5: Invariants checked.

ID	TLA+ Operator	Description	TLC	Lean
INV-1	`NoEmptyLevels`	Every price level has a non-empty order queue	trivial	trivial
INV-2	(sortedness)	Bids strictly descending	by constr.	✓
INV-3	(sortedness)	Asks strictly ascending	by constr.	✓
INV-4	`BookUncrossed`	$bestBid < bestAsk$ or either side empty	✓	✓
INV-5	`NoGhosts`	Every resting order has $remainingQty > 0$	✓	—
INV-6	`StatusConsistency`	Resting orders have status NEW or PARTIAL	✓	—
INV-7	`FIFOWithinLevel`	FIFO within level (strictly increasing timestamps)	fixed*	—
INV-8	`NoRestingMarkets`	MARKET orders never appear on the book	✓	—
INV-9	(construction)	Trade price equals the passive order's price	by constr.	by constr.
INV-11	`PostOnlyGuarantee`	No trade where the aggressor was post-only	✓	—
INV-12	`STPGuarantee`	No trade between orders in the same STP group	✓	—
INV-13	`NoRestingMTL`	MTL orders are converted before resting	✓	—
INV-14	`NoRestingMinQty`	*minQty* is cleared before an order rests	✓	—

IDs match the canonical §13 summary of the prose specification. TLC column: ✓ means bounded exhaustive pass across all configurations in Table 4. Lean column: ✓ means part of `AllInv` and mechanically proved for all book sizes by `process_preserves_AllInv`; — means not yet lifted to Lean. *INV-7 was violated under the original specification and passes after the fix described in §5.1.

5 Findings and Counterexamples

5.1 Finding 1: FIFO Violation via Missing Timestamp on Triggered Stops

Severity. High—directly violates INV-7 (FIFO within price level).

Root cause location. §10.1 `EVALUATE_STOPS` interaction with §6.1 `INSERT`.

The Mechanism

The prose specification's stop-trigger procedure (§10.1) converts a dormant stop order to its base type (LIMIT or MARKET) and re-enters it into the `PROCESS` pipeline. The stop order's `timestamp` field is not updated at trigger time. It retains the timestamp assigned when the order was originally submitted and placed into dormancy.

When the triggered LIMIT order does not fully fill and rests on the book via `DISPOSE` (§5.2), it is appended to the back of the appropriate price-level queue per §6.1—giving it the lowest priority at that level. However, `FIFOWithinLevel` requires that the newly appended order have a timestamp strictly greater than all orders already in the queue. Because orders already in the queue arrived after the stop was submitted—they carry later timestamps—the stop's original timestamp is numerically earlier. The queue is now out of order.

Counterexample Trace

TLC produced the following minimal counterexample. Times shown are logical clock values.

```
Step 1: Submit SELL LIMIT  qty=1, price=1      (id=1, ts=1)
        -> Rests on ask:  askQ[1] = [order1(ts=1)]

Step 2: Submit BUY STOP_LIMIT  stop=1, price=1, qty=1  (id=2, ts=2)
        -> lastTradePrice = NULL -> does not trigger
        -> Placed in stops = {order2(ts=2)}

Step 3: Submit BUY LIMIT  qty=2, price=1       (id=3, ts=3)
        -> Matches askQ[1]: fills order1 fully (fillQty=1)
        -> Trade: price=1, aggressor=3, passive=1
        -> lastTradePrice = 1
        -> order3: remainingQty=1, rests on bid
           bidQ[1] = [order3(ts=3)]
        -> Stop cascade: lastTradePrice(1) >= stopPrice(1)
           -> order2 triggers, converts to BUY LIMIT
           -> Timestamp NOT updated: ts=2 (STALE)
           -> Ask side empty, no fill
           -> Dispose: order2 rests on bid
              bidQ[1] = [order3(ts=3), order2(ts=2)]

        FIFOWithinLevel check:
            bidQ[1][1].timestamp = 3
            bidQ[1][2].timestamp = 2
            3 < 2 ?  FALSE  ->  INVARIANT VIOLATED
```

Figure 1: TLC counterexample trace for the FIFO violation. The triggered stop order (id=2) retains its original timestamp (ts=2) and is appended behind an order with a later timestamp (ts=3), producing a non-monotonic queue.

Why Manual Review Missed This

The prose specification treats stop triggering and order insertion as separate sections (§10.1 and §6.1 respectively). Each section is locally correct: §10.1 correctly describes conversion, and §6.1 correctly describes insertion. The interaction—that converted stops carry stale timestamps—is invisible when reading either section in isolation. A reviewer must simultaneously hold the conversion path, the insertion path, and the FIFO invariant in mind to see the problem. TLC holds all three simultaneously by construction.

5.2 Finding 2: Iceberg Stranding Under STP DECREMENT

Severity. Medium—creates a reachable book state with invalid order geometry.

Root cause location. §8.3 DECREMENT case; missing interaction with §7.5 iceberg reload.

The Mechanism

The STP DECREMENT policy reduces both the incoming order's *remainingQty* and the resting order's *visibleQty* and *remainingQty* by min(*incoming.remainingQty*, *resting.visibleQty*), without generating a trade. The iceberg reload logic (§7.5 step 3) specifies:

> When *visibleQty* = 0 and *remainingQty* > 0, reload the visible slice, assign a new timestamp, and move the order to the back of the queue.

However, this reload is only specified inside the fill path. The DECREMENT action (§8.3) does not reference §7.5. There is no clause in §8.3 for the case where DECREMENT drives *visibleQty* to zero while *remainingQty* > 0.

The result is a resting iceberg order with *visibleQty* $= 0$ and *remainingQty* > 0. It cannot be matched—$\mathit{fillQty} = \min(\mathit{incoming.remainingQty}, 0) = 0$—and cannot reload because the reload trigger is only in the fill path. The order is stranded: it occupies a queue position, blocks no fills, and cannot be removed except by explicit cancel.

Illustrative Scenario

```
Resting iceberg: qty=2, displayQty=1, visibleQty=1,
                 remainingQty=2, stpGroup="G1"        (id=R)
Incoming:        qty=1, stpGroup="G1", stpPolicy=DECREMENT  (id=I)

DECREMENT:
    reduceQty = min(1, 1) = 1
    I.remainingQty = 0   -> I is cancelled
    R.visibleQty   = 0
    R.remainingQty = 1   -> R remains on book

Post-decrement state of R:
    visibleQty=0, remainingQty=1, displayQty=1
    -> Reload path not invoked
    -> R sits on book, invisible, unreachable by matching
```

Note on claim level. This scenario is reachable in the prose specification as written. However, the TLA+ model as checked incorporates the fix (iceberg reload after DECREMENT) because the stranded state would cause the model checker itself to loop: subsequent orders would find *visibleQty* $= 0$ and skip the order without decrementing fuel, creating an effective infinite loop in `DoMatch`. The bug is therefore presented as a specification gap—a path the spec silently omits—rather than as a TLC-witnessed invariant violation with a precise counterexample trace.

5.3 Design Observations (Not Demonstrated Bugs)

The following interactions are noted as potential specification ambiguities. They are not presented as verified bugs because no invariant violation was produced for them.

Observation 1—FOK + STP DECREMENT. The FOK pre-check counts non-conflicting visible quantity. During matching, DECREMENT can consume the incoming order's *remainingQty* without generating trades. The incoming order reaches *remainingQty* $= 0$ and is treated as cancelled, but with fewer trade fills than *qty*. Whether this constitutes a FOK violation depends on whether FOK is defined as "fully traded" or "*remainingQty* fully consumed." The spec does not address this interaction.

Observation 2—MinQty + STP DECREMENT. Analogously, the MinQty pre-check guarantees *filled_quantity* $\geq$ *minQty* (§5.4), but if DECREMENT consumes part of the incoming quantity before any fills occur, the traded quantity may fall below *minQty* even though the pre-check passed.

Observation 3—MTL + MinQty clearing. The spec clears *minQty* in two places: once in the MTL path (Phase 4) and once in the normal matching path (Phase 5a). This duplication

is currently correct but fragile—adding a new order type or routing path requires remembering to add a third clearing point.

6 Specification Fixes

Both confirmed issues were fixed in the TLA+ model. The fixes are minimal and directly address the root causes.

Table 6: Bug and fix summary.

ID	Location	Description	Fix	Invariant
BUG-1	§10.1 `EVALUATE_STOPS`	Triggered stop retains original timestamp; violates FIFO when resting	Assign `stop.timestamp = currentTimestamp()` before re-entering pipeline	INV-7
BUG-2	§8.3 DECREMENT	Iceberg *visibleQty* driven to 0 without reload; order stranded	After DECREMENT: if $visibleQty = 0 \wedge remainingQty > 0 \wedge displayQty \neq \perp$, reload visible slice, refresh timestamp, move to back	Spec gap

6.1 Fix for BUG-1 (Stop Timestamp)

The fix is a one-line addition to §10.1. In the TLA+ model, `ProcessTriggeredStops` applies the fix:

```
converted == [ConvertStop(s) EXCEPT !.timestamp = tm]
```

The corresponding prose amendment to §10.1:

```
-- After "Convert to base type":
stop.timestamp = currentTimestamp()   -- Assign trigger-time priority
PROCESS(stop, book)
```

This ensures that a triggered stop order receives a timestamp reflecting when it entered the live matching pipeline, not when it was originally submitted.

6.2 Fix for BUG-2 (Iceberg Reload After DECREMENT)

The fix adds a post-DECREMENT iceberg check to §8.3. In the TLA+ model:

```
ELSE IF newRest.visibleQty = 0 ∧ newRest.displayQty /= NULL THEN
    LET reloadQty == Min(newRest.displayQty, newRest.remainingQty)
        reloaded  == [newRest EXCEPT !.visibleQty = reloadQty,
                                     !.timestamp = tm,
                                     !.status = "PARTIAL"]
        sc == SetContra(side, bQ, aQ, bestP,
                        restTail \o ⟨reloaded⟩)
    IN DoMatch(newInc, sc.bQ, sc.aQ, tds, tm + 1, fuel - 1)
```

The corresponding prose amendment to §8.3, after the DECREMENT quantity update:

```
IF resting.visibleQty = 0 AND resting.remainingQty > 0
   AND resting.displayQty != null:
```

```
    resting.visibleQty = min(resting.displayQty, resting.remainingQty)
    resting.timestamp  = currentTimestamp()
    MOVE resting TO back OF level.orders
```

This mirrors the iceberg reload behavior specified in §7.5 for the normal fill path, extending it to the DECREMENT path for consistency.

7 Lean 4 Mechanized Proof

TLC searches a bounded state space and cannot establish correctness for arbitrary book sizes. Production matching engines routinely hold millions of orders, well beyond the configurations TLC can enumerate. To lift the argument above that bound we translated the fixed specification into a Lean 4 reference implementation and mechanized a proof of the principal preservation property. This section states the theorem, describes an early soundness issue with a constant fuel bound, and presents two complementary proof artifacts: a long constructive proof of the full preservation theorem by branch-by-branch induction and a shorter structural proof of the matching-and-dispose core by case analysis of the matching termination condition.

Both proof files live in the same Lean project as the executable matching engine and are type-checked on every build, with no open obligations remaining. Table 7 summarises their sizes.

Table 7: Lean 4 proof artifacts (final state).

File	Lines	Theorems	Proof style
`Theorems.lean`	3,986	74	Constructive, branch-by-branch
`TheoremsElegant.lean`	312	6	Three-case trichotomy
Total	**4,298**	**80**	Complementary proof artifacts

7.1 Goal and Statement

Define the combined book invariant

$$\texttt{AllInv}(b) \equiv \texttt{BookUncrossed}(b) \wedge \texttt{bidsSortedDesc}(b) \wedge \texttt{asksSortedAsc}(b).$$

The principal result we establish is that `AllInv` is preserved by the full processing pipeline under a minimal well-formedness precondition. In Lean it is stated first as a mutual induction over `processOrder`, `processCascade`, and `processTriggeredStops`; the exported single-order theorem is:

```
theorem processOrder_preserves_AllInv
    (fuel : Nat) (o : Order) (b : BookState)
    (hpok : OrderProcOk o)
    (hstops : StopsNoPostOnly b)
    (h : AllInv b) :
    AllInv (processOrder fuel o b).book
```

The corresponding `process`-level `AllInv` corollary is immediate because `process` only adds metadata updates (`id`, `timestamp`, `nextId`, `clock`) and does not change bids or asks. We state the uncrossed-book projection separately under the name `process_preserves_uncrossed` because it is the property most directly comparable to the TLC invariant `BookUncrossed`, but the underlying proved result is the stronger `AllInv` preservation statement.

The sortedness conjuncts of `AllInv` are not cosmetic. `BookUncrossed` alone is not inductive with respect to the matching loop: the proof does not close unless the induction hypothesis also

records that both sides of the book are monotone in price. This was established by the induction refusing to close under the weaker hypothesis, at which point the invariant was strengthened to `AllInv` and the proof proceeded. We report this as an example of the proof refining the invariant the specification should have emphasized.

The preconditions `OrderProcOk` and `StopsNoPostOnly` are not arbitrary hypotheses: they are minimal fragments of the specification's own well-formedness rules (WF-13: post-only ⇒ LIMIT, and the derived LIMIT ⇒ price defined), also surfaced by the proof itself. We return to them in §7.6.

7.2 The Fuel Problem

Lean 4's termination checker requires every recursive function to be provably total. The matching loop `doMatch` is recursive: one step consumes a resting order, reduces the incoming order's quantity, or removes a crossing price level, and recurses on the remainder. Because the Lean termination checker cannot synthesize the measure automatically, the original implementation used an explicit `fuel : Nat` argument and a *constant* bound:

```
def defaultFuel : Nat := 100
def doMatch (fuel : Nat) ... : MatchResult :=
  match fuel with
  | 0       => /* fuel exhausted: return as-is */
  | fuel'+1 => /* one step, recurse with fuel' */
```

This hides the assumption that matching always completes within 100 steps, which does not hold at arbitrary scale. Consider a book with 200 resting asks at price 50 and one incoming BUY at price 100. `doMatch` consumes the first 100 asks, exhausts its fuel, and returns the incoming order with 100 asks still resting at price 50. The disposition phase then inserts the buy at price 100 on the bid side, producing $bestBid = 100 > bestAsk = 50$—a crossed book. The preservation theorem does not hold under the constant-fuel assumption, and the proof fails to close.

This is the point at which the difference between the two verification levels becomes concrete. TLC's explored configurations never exceeded a handful of orders on either side, so the bounded model-checking pass did not expose the issue; only the universally quantified Lean statement forced the fuel bound to become a proof obligation rather than an assumption. The arbitrary constant turned a semantic property—"matching completes"—into a syntactic one—"matching completes in ≤ 100 steps"—and only the syntactic version was provable.

7.3 Computed Fuel and the Progress Lemma

The fix for the fuel unsoundness is to compute the fuel bound from the book state and prove that the bound is always sufficient:

```
def matchMeasure (contra : List PriceLevel) (_inc : Order) : Nat :=
    totalRemaining contra + orderCount contra + contra.length

def computeMatchFuel (b : BookState) (side : Side) : Nat :=
    matchMeasure (contraLevels b side) dummyInc + 1
```

The measure has three components: `totalRemaining` $= \sum_{o \in \text{contra}} o.remainingQty$ accounts for fills, `orderCount` accounts for orders whose visible quantity is skipped without a fill (STP cancellation, zero-visible iceberg slots), and `contra.length` accounts for level exhaustion steps that remove an empty price level.

The key technical lemma—the *progress lemma*—is that every non-terminal step of `doMatch` strictly decreases this measure. A single monolithic induction over all of `doMatch`'s branches (there are 14) was intractable in our experience. The decomposition that worked is per-branch:

each branch becomes its own 10–25-line lemma proving progress, and the 14 lemmas compose into a single termination-and-stability theorem. Table 8 summarizes the branches and the decrement each one contributes.

Table 8: Progress analysis for `doMatch` branches. Terminal branches stop the loop; measure-decreasing branches recurse on a strictly smaller state.

Branch	Kind	Why the measure decreases
Full fill (resting consumed)	Decrease	`orderCount` drops by 1
Partial fill (incoming done)	Terminal	$remainingQty = 0$
STP cancel newest	Terminal	incoming cancelled
STP cancel oldest	Decrease	one order removed
STP cancel both	Terminal	incoming cancelled
STP decrement	Decrease	resting qty strictly reduced
Iceberg fill + reload	Decrease	fill reduced `totalRemaining` before reload
Level exhausted	Decrease	empty level removed
Zero-visible skip	Decrease	one order removed
No crossable orders	Terminal	¬ `canMatchPrice`

The iceberg reload case is the subtlest. Reload appears to "add liquidity back": the resting order's visible quantity jumps from zero to the display quantity. But `totalRemaining` is not affected by this redistribution—only the visible/hidden split changes—and the fill step that triggered the reload strictly decreased `totalRemaining` by the filled quantity. The composition of the two steps is a net decrease. The proof carries this through explicitly; the reload is not a no-op, it is a book-keeping update that leaves the measure strictly below its pre-step value.

With the progress lemma in place, the terminal states of `doMatch` (on sufficient fuel) fall into a four-way stable predicate: the incoming is consumed, the incoming is cancelled, the contra side is empty, or the contra head does not cross the incoming price. This predicate (`buyMatchStable` / `sellMatchStable`) is the black-box characterization that the elegant proof in §7.7 relies on.

7.4 Side Parameterization

Every matching lemma has both a buy-side and a sell-side form. The first attempt duplicated every proof—`doMatch_buy_preserves_bids` alongside `doMatch_sell_preserves_asks`, and so on for fourteen branches on each side. The duplication doubled the proof effort and made any change require two coordinated edits.

The fix is to parameterize every lemma by a `Side` value:

```
theorem doMatch_preserves_uncrossed
    (fuel : Nat) (inc : Order) (b : BookState)
    (tm : Timestamp) (side : Side)
    (hside : inc.side = side)
    (h : AllInv b) :
    BookUncrossed { b with
        bids := (doMatch fuel inc b.bids b.asks [] tm).bids,
        asks := (doMatch fuel inc b.bids b.asks [] tm).asks }
```

The proof then performs `cases side` *only at the leaves* where the concrete direction of the inequality matters. Everything above the leaves is shared. After side-parameterization the sell mirror of the fuel-sufficiency proof—which had been the bottleneck—compiled on the first try.

7.5 Mutual Induction Across the Cascade

`processOrder`, `processCascade`, and `processTriggeredStops` are mutually recursive: a trade can trigger a dormant stop, which converts to a market order and re-enters the pipeline, which can produce more trades, which can trigger more stops. Proving `AllInv` preservation for any one of them requires proving it for all three simultaneously:

```
theorem process_all_preserve_AllInv : forall fuel,
  (forall o b, OrderProcOk o -> StopsNoPostOnly b -> AllInv b ->
    AllInv (processOrder fuel o b).book) /\
  (forall ts b, StopsNoPostOnly b -> AllInv b ->
    AllInv (processCascade fuel ts b).book) /\
  (forall os b, (forall o in os, o.postOnly = false) ->
                StopsNoPostOnly b -> AllInv b ->
    AllInv (processTriggeredStops fuel os b).book)
```

Induction proceeds on `fuel`. The base case (`fuel = 0`) is trivial because all three functions return the book unchanged. The step case must discharge every phase of `processOrder`: stop handling, post-only rejection, FOK pre-check, MinQty pre-check, market-to-limit routing, normal matching, dispose, and stop cascade. Each phase follows the same template: apply `doMatch_preserves_AllInv` to the matching step, extract the non-crossing precondition from the termination-stability predicate, apply `dispose_preserves_AllInv` to the disposition step, and thread the cascade through the inductive hypothesis for `processCascade`.

The subtlest sub-case is *MTL Phase 4 case 3*: a MARKET-TO-LIMIT order that fills partially, converts to a LIMIT order, then runs a *second* matching pass with the derived limit price, then disposes. The second `doMatch` operates on the post-first-match book with a freshly constructed incoming order, and the non-crossing precondition for the subsequent dispose must be extracted from a `doMatch` call that does not match the form of any of the existing helper lemmas. We resolved this by parameterizing the non-crossing extraction helpers by the starting timestamp (`matching_dispose_noCross_buy` / `sell` each take an explicit `tm`) so they apply uniformly to both the first matching pass (which uses `b.clock + 1`) and the MTL second pass (which uses `mr.clock`).

7.6 Minimal Well-Formedness Preconditions

The precondition `OrderProcOk o` captures two facts that the proof cannot avoid:

```
def OrderProcOk (o : Order) : Prop :=
    (o.postOnly = true -> o.price.isSome = true) /\
    ((o.orderType = .stopLimit \/ o.orderType = .stopMarket) ->
       o.postOnly = false)
```

The first clause rules out the post-only / no-price configuration that the spec forbids via WF-13 (*post-only* $\Rightarrow$ *LIMIT*) and WF-2 (*LIMIT* $\Rightarrow$ *price defined*). In the proof, this hypothesis discharges the otherwise-unreachable branch in which `wouldCross` returns false for a post-only order with `price = none`. Without it, the Phase-2 case of `processOrder` has a genuinely crossed output.

The second clause is a companion invariant `StopsNoPostOnly b` $\equiv \forall s \in b.stops,\ s.postOnly =$ `false`, which is preserved by every pipeline phase (none of them introduce a new stop order with `postOnly = true`) and is used by the stop-cascade case.

Both preconditions were *discovered*, not guessed. We first attempted the proof without them; the proof failed at a specific branch with a concrete counterexample; the counterexample corresponded exactly to a state that the spec's own WF rules forbid. The fragment we added is strictly weaker than the full `Order.wellFormed` predicate—we only require what the proof actually needs. In particular, we do *not* require the incoming order to pass the full twenty-

constraint WF check; only the two clauses above. This is the honest precondition: the proof tells us what matters.

7.7 The Elegant Alternative Proof

Once the constructive proof was complete, we developed a second proof component, in the file `TheoremsElegant.lean` (6 theorems, 312 lines), that recasts the matching-and-dispose core as a shorter structural argument.

When `doMatch` terminates on a fuel-sufficient run, the resulting state falls into one of three cases:

1. **Aggressor consumed.** $remainingQty = 0$ (or the order has been cancelled). The incoming order is gone; `doMatch` only removes orders from the contra side, never adds to it; removing orders cannot create a cross.
2. **Contra side empty.** There is nothing left on the opposite side to cross against; an empty side is vacuously non-crossing.
3. **Non-crossing head.** Both sides are non-empty but the contra head does *not* cross the incoming price. This is the only way `doMatch` can stop with both sides active: if prices crossed, the loop guard `canMatchPrice` would fire and matching would take another step. Since it stopped, prices do not cross.

A fourth case—both sides active and still crossing—is not reachable at termination: such a state is not terminal under the loop guard, so fuel sufficiency rules it out. The three cases compose with `dispose` directly: Case 1 returns the book unchanged, Case 2 inserts on a book whose contra side is empty (vacuously non-crossing), and Case 3 inserts at a price strictly separated from the contra head.

The second proof treats `doMatch` as a black box characterised by its termination condition rather than by the control-flow path that produced it. It reuses the constructive proof's fuel-sufficiency lemma (`doMatch_output_stable`) as an imported fact.

	Constructive	Structural
File	`Theorems.lean`	`TheoremsElegant.lean`
Lines	3,986	312
Theorems/lemmas	74	6
Argument	Branch-by-branch	Three-case analysis

Both are machine-checked, and both live in the build. The constructive proof discharges fuel sufficiency, sortedness preservation, side isolation, and the mutual pipeline theorem. The structural file reuses those lemmas and shows that, once the termination condition is fixed, the matching-and-dispose argument itself is short.

7.8 Model Checking and Theorem Proving Are Complementary

TLC and Lean surfaced different classes of issue. TLC produced the two feature-composition defects (FIFO via stale timestamps; iceberg stranding under STP DECREMENT) as concrete multi-step traces that a proof attempt would not have generated directly. Lean's universal quantification, in turn, made the constant-fuel assumption (§7.2) visible; TLC did not reach scopes where the fuel bound mattered.

At the level of cost, TLC returned its first counterexample in 102 seconds of wall time, while the Lean proof accumulated weeks of interactive development and produced a 4,298-line artifact. The two are complementary rather than substitutable: TLC's speed is useful early, and the Lean statement scales beyond any bounded configuration.

8 Specification-Guided Implementation

For completeness we describe how the fixed specification was carried forward into systems-language implementations. We frame this as an engineering observation, not as an additional verification result: we do not prove refinement between the Lean reference and the C or C++ code, and the assurance we obtain at the implementation level is bounded by shared tests and runtime invariant checks rather than by a mechanized proof.

Prose and TLA+ artifacts from §2–§6 were used as the input to a language-model-assisted coding workflow, augmented with two architectural directives: intrusive doubly-linked lists for $O(1)$ order removal and FIFO management, and a pool allocator to bound allocation latency. The resulting C and C++ implementations follow the phase structure of the specification one for one, and apply the two defect fixes introduced in §6: stop-trigger timestamps are refreshed before re-entry into the matching pipeline, and the STP DECREMENT path reloads iceberg visible quantity when it would otherwise reach zero.

The C implementation additionally includes an `engine_check_invariants` routine that translates the TLC-checked invariants into runtime assertions, giving a form of continuous conformance checking between the running engine and the invariant suite. This is weaker than a refinement proof—an invariant assertion fires after state has already diverged—but it narrows the gap between specification and implementation and has caught regressions during development.

We deliberately avoid framing this step as "correct-by-construction" code generation. A precise specification is an unusually good input for an implementation effort, and a feature-rich matching engine is an unusually tractable target because its control-flow structure is exposed by the specification itself. The two defect fixes were carried across because the specification was explicit about them, not because the generation step carried any inherent guarantee. A genuine end-to-end guarantee would require a refinement argument connecting the Lean reference to the systems-language code, and that is beyond the scope of this paper.

9 Lessons Learned

9.1 Feature Composition Is Where Bugs Hide

Both findings share a structural pattern: each involved feature is correctly specified in isolation, but the specification omits a clause covering their interaction. Stop orders are correctly described; iceberg reload is correctly described; FIFO insertion is correctly described. None of these sections contains an error when read alone. The bug lives in the intersection: what happens to an iceberg order's timestamp when the mechanism that zeros its visible quantity is not the fill path but the STP DECREMENT path?

This is not a failure of careful writing—it is a structural limitation of prose specifications. Prose sections have natural scope boundaries. Interactions between features that span sections are systematically harder to see than behaviors within a single section.

A formal model collapses all features into a single operational semantics and evaluates invariants globally. The intersection cases are not special—they are just more states.

9.2 Logical Clocks Amplify Ordering Bugs

The FIFO invariant is stated in terms of timestamps. Because the specification uses a monotonically increasing logical clock, any state transition that fails to advance the clock before inserting an order can produce a stale timestamp that violates ordering. The stop-trigger bug is exactly this: the clock was not advanced before the triggered stop was assigned to the book.

Timestamp-based ordering invariants are fragile in the presence of features (like stop triggers) that re-enter orders into the pipeline from a dormant pool. Wherever an order exits a

dormant state and re-enters an active queue, its timestamp must be refreshed. The fix is simple; the risk is that prose specifications do not enumerate every re-entry point.

9.3 Iceberg Reload Is Not Tied Tightly Enough to Its Trigger Condition

The iceberg reload semantics in §7.5 are described in terms of the fill path: "when *visibleQty* is fully consumed [by a fill]." The prose implicitly assumes that *visibleQty* can only reach zero through the fill path. STP DECREMENT provides a second path to zero *visibleQty* that the reload trigger does not anticipate.

A more robust specification would state the reload condition declaratively as an invariant on the resting order state:

> A resting order must not have $visibleQty = 0$ and $remainingQty > 0$.

Rather than specifying reload as a step triggered by the fill path, it would be specified as a consequence of this invariant: any action that could produce this state must be followed by a reload. This declarative framing would make the omission in DECREMENT immediately apparent.

9.4 The Specification Artifact Has Independent Value

The TLA+ model serves as a precise, executable reference for the prose specification. Independently of the bugs found, it forces every semantic choice to be made explicit: what is the recursion structure of stop cascades? What exactly does "timestamp" mean for a triggered order? At what point does MTL price derivation occur relative to the cascade? These questions have definite answers in the model even when the prose is ambiguous.

10 Limitations and Reproducibility

10.1 Bounded Exhaustive Exploration, Not Proof (for TLA+)

TLC performs bounded exhaustive state enumeration: it finds every state reachable from `Init` via the `Next` relation, within the parameter bounds. A clean run means no invariant violation was found within the explored configurations. It does not mean no violation exists for larger configurations, more orders, or more price levels. We do not claim the TLA+ model is correct in any unbounded sense.

Specifically:

- The 3-order partial run did not complete state-space exploration.
- No configuration with more than two distinct STP groups was checked.
- GTD time-based expiry was checked only via the simplified `TimeAdvance` action, not against wall-clock constraints.
- The event-ordering invariant (INV-10) is not formalized in the model.

Lean closes the unbounded gap for `AllInv`, not for the full invariant suite. The Lean proof in §7 establishes `process_preserves_AllInv` for arbitrary book sizes and arbitrary order counts, under the minimal preconditions `OrderProcOk` and `StopsNoPostOnly`. The corollary `process_preserves_uncrossed` is an immediate consequence. Among the broader TLC-checked invariants, only the `AllInv` portion (INV-2, INV-3, INV-4) has been lifted to Lean. The remaining invariants—no ghosts (INV-5), status consistency (INV-6), FIFO within level (INV-7), no resting markets (INV-8), post-only safety (INV-11), STP guarantee (INV-12), no resting MTL (INV-13), no resting MinQty (INV-14)—remain at the bounded level of assurance provided by TLC. Lifting them to Lean is future work.

10.2 Specification vs. Implementation

For the Lean reference implementation, the gap is closed. The Lean 4 code that `#eval` runs in tests is the same code that the theorems reference—there is no separate model, no extraction layer, and no refinement proof obligation. For the Lean artifact, the semantic gap between tested behavior and proved behavior is zero.

For the C and C++ synthesized implementations, a gap remains. The Lean proof does not transfer automatically to C or C++; those implementations are validated by shared test suites derived from the specification, not by a formal correspondence argument. An implementation that correctly implements the (fixed) specification could still contain implementation bugs—off-by-one errors, concurrency hazards, data-structure corruption—that are outside the scope of this work. A refinement proof connecting the Lean reference to the systems-language implementations would close the remaining gap and is a natural next step.

10.3 Reproducibility

All artifacts described in this paper—the prose specification, the TLA+ model and configurations, the Lean 4 project, and the paper source—are available in a public Git repository at https://github.com/formally-verified-exchange/verified-matching-engine. The top-level `README.md` lists prerequisites and gives step-by-step commands for building the Lean proofs and running TLC against each configuration. The complete artifacts are:

- `matching-engine-formal-spec.md`—the prose specification.
- `spec/MatchingEngine.tla`—the TLA+ specification (∼950 lines).
- `spec/MatchingEngine.cfg` and the per-configuration files `spec/MatchingEngine_*.cfg` for Table 4.
- `spec/REPORT.md`—model checking output and bug descriptions.
- `MatchingEngine/`—the Lean 4 project (11 source files). The two proof files are `Theorems.lean` (3,986 lines, 74 theorems) and `TheoremsElegant.lean` (312 lines, 6 theorems), both under the `MatchingEngine/` directory. Both are built and type-checked by `lake build` and are imported from the library root `MatchingEngine.lean`.
- Synthesized C/C++ implementations (available in related repositories).

To reproduce the FIFO counterexample:

1. Revert the stop timestamp fix in `ProcessTriggeredStops`: replace `[ConvertStop(s) EXCEPT !.timestamp = tm]` with `ConvertStop(s)`.
2. Run TLC with the Small configuration (`MAX_ORDERS=2, MAX_QTY=2, PRICES={1,2}`, amend disabled).
3. TLC will find a violation of `FIFOWithinLevel` within the first several million states.

To reproduce the fuel-unsoundness counterexample (§7.2):

1. Revert `computeMatchFuel` in `MatchingEngine/Process.lean` to the constant `defaultFuel := 100`.
2. Re-run `lake build`. The proof of `doMatch_preserves_AllInv` will fail: the measure argument no longer dominates the recursive call bound, and any book with more than 100 contra-side orders produces a concrete cross after one application of `process`.

11 Related Work

The work we know of falls into three largely disjoint literatures: formal specification and model checking of distributed systems, mechanized proof of systems software, and economic analysis of exchange microstructure. Each addresses part of what this paper covers, but none combines a feature-rich limit-order-book specification with both bounded bug-finding and mechanized theorem-level assurance. The positioning below maps the closest neighbors.

TLA+ in industry. TLA+ has been applied at scale at Amazon Web Services [1], where engineers used it to model DynamoDB, S3, and other distributed services and uncovered design-level bugs that would have been difficult to find through testing. Similar reports have come from Microsoft (Azure Cosmos DB), MongoDB, and Elastic. Our work follows the Newcombe et al. template—use TLC to explore a specification's state space and find interaction bugs before coding—but differs in the target domain (a financial matching engine rather than a distributed storage protocol) and in the follow-through to a machine-checked proof in a different tool (Lean 4) rather than stopping at bounded model checking.

Mechanized proof of systems software. The largest-scale machine-checked software artifacts—the seL4 microkernel [2], CompCert [3], and CertiKOS [4]—prove functional correctness or refinement between a high-level specification and a C-level implementation, and consumed person-decades of effort. The present work is considerably narrower in both ambition and cost: we prove preservation of a single book invariant (`AllInv`) for the Lean reference implementation of a feature-rich matching engine and rely on a separate testing-and-assertion path to connect that reference to systems-language code. The investment is correspondingly smaller (weeks rather than years), but the target is a different class of system—one whose complexity lives in feature composition rather than in concurrency or low-level memory semantics.

Formal methods for financial systems. Prior formal work on financial infrastructure has focused mostly on auction-theoretic properties of matching algorithms [5] and on smart-contract verification for decentralized exchanges [6, 7]. Property-based testing tools like QuickCheck [8] are used at some trading firms for engine validation, but published accounts of end-to-end formal verification of a production-style limit-order-book engine with the full order-type suite (icebergs, stops, STP, post-only, FOK, MinQty) are, to our knowledge, absent from the literature. A TLA+ specification of an order book has been circulated informally in the TLA+ user community, but with a substantially smaller feature set and no mechanized proof layer.

Termination proofs and well-founded recursion. The core technical device in §7.3—replacing an arbitrary fuel bound with a state-derived measure and a progress lemma—is a standard pattern in mechanized proofs of imperative programs [9, 10]. Our contribution is not the pattern itself but a concrete demonstration that an LLM-assisted proof can be pushed through the per-branch progress argument for a matching-loop implementation with 14 interacting branches, closing the bug introduced by the arbitrary constant.

LLM-assisted formal verification. A growing body of work explores the use of large language models to draft Lean, Coq, or Isabelle tactics [11, 12]. These systems focus on automating tactic search. Our experience is complementary: the AI did not autonomously close the proof through tactic search; it wrote naturalistic Lean proofs under human strategic direction. The progress-lemma decomposition, the side-parameterization refactor, and the three-case trichotomy were human contributions. The 4,000 lines of case-splitting and omega invocations were the AI's contribution. Neither layer alone would have produced the artifact.

Spec-to-implementation synthesis. Program synthesis from formal specifications is a classical research area [13]. Recent work on LLM-driven synthesis [14] has reached practical levels for small programs. Our "Guided Synthesis" paradigm differs in that the specification is both the logical source of truth *and* the prompt, and architectural hints (intrusive lists, pool allocation) are provided as orthogonal performance directives. The result is then validated for conformance via shared test vectors derived from the spec. A formal refinement argument between the Lean reference and the synthesized C/C++ is future work.

12 Conclusion

We formalized a feature-rich matching-engine specification in TLA+ and ran bounded exhaustive exploration across several configurations totaling over 36 million distinct states. TLC identified one specification defect—a FIFO violation caused by a triggered stop order retaining its original submission timestamp—and one specification gap in the interaction between STP DECREMENT and iceberg visibility. The FIFO counterexample is a three-order sequence whose minimality is part of the finding: reading the stop, insertion, and FIFO sections of the prose specification individually does not expose it, but an operational model that composes all three does.

We then proved, in Lean 4, that the processing pipeline preserves the combined book invariant `AllInv` (uncrossed plus sorted on both sides) under a minimal well-formedness precondition, for arbitrary book sizes. The matching-loop fuel bound is derived from the book state rather than assumed as a constant. `Theorems.lean` (3,986 lines, 74 theorems) proves the full preservation theorem branch-by-branch through the 14 branches of the matching loop and the 11 phases of `processOrder`. `TheoremsElegant.lean` (312 lines, 6 theorems) adds a shorter structural argument for the matching-and-dispose core via a three-case analysis of the matching termination condition. Both files are type-checked by every build and contain no open obligations.

One issue surfaced during the proof effort itself: the original arbitrary fuel bound made the preservation statement false for sufficiently populated books. Replacing it with a state-derived measure and a progress lemma restores soundness.

The broader takeaway is that bounded model checking and mechanized theorem proving play complementary roles on the same specification. TLC is effective at surfacing concrete feature-composition defects inside small state bounds and at speeds useful during early specification work. Lean provides guarantees that hold for arbitrary scale and, as a side effect, exposes pipeline issues that bounded exploration cannot reach. Neither tool substitutes for the other.

Acknowledgments

The TLA+ model, the Lean 4 proofs, and the C/C++ implementations discussed in §8 were developed interactively with the assistance of a large language model, using the formal specification as the primary input. The constant-fuel soundness issue described in §7 was encountered during this process.

References

[1] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. How Amazon Web Services Uses Formal Methods. *Commun. ACM*, 58(4):66–73, 2015.

[2] G. Klein et al. seL4: Formal Verification of an OS Kernel. In *Proc. ACM SOSP*, 2009.

[3] X. Leroy. Formal Verification of a Realistic Compiler. *Commun. ACM*, 52(7):107–115, 2009.

[4] R. Gu et al. CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels. In *Proc. USENIX OSDI*, 2016.

[5] E. Budish, P. Cramton, and J. Shim. The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response. *Quarterly Journal of Economics*, 130(4):1547–1621, 2015.

[6] K. Bhargavan et al. Formal Verification of Smart Contracts: Short Paper. In *Proc. PLAS*, 2016.

[7] I. Grishchenko, M. Maffei, and C. Schneidewind. A Semantic Framework for the Security Analysis of Ethereum Smart Contracts. In *Proc. POST*, 2018.

[8] K. Claessen and J. Hughes. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proc. ACM ICFP*, 2000.

[9] P. Manolios and D. Vroon. Termination Analysis with Calling Context Graphs. In *Proc. CAV*, 2006.

[10] K. R. M. Leino. Developing Verified Programs with Dafny. In *Proc. ICSE*, 2013.

[11] K. Yang et al. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Proc. NeurIPS*, 2023.

[12] E. First, M. Rabe, T. Ringer, and Y. Brun. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *Proc. ESEC/FSE*, 2023.

[13] Z. Manna and R. Waldinger. A Deductive Approach to Program Synthesis. *ACM TOPLAS*, 2(1):90–121, 1980.

[14] N. Jain et al. Jigsaw: Large Language Models Meet Program Synthesis. In *Proc. ICSE*, 2022.